

Resumen teórico — Sistemas Operativos

Notas del curso: *Gestión de la memoria*

Tomás Rodeghiero

Sistemas Operativos — Universidad Nacional de Río Cuarto

2026

Resumen

Este documento es un resumen teórico del capítulo *Gestión de la memoria* de las *Notas del curso* de Sistemas Operativos (Marcelo Arroyo, UNRC). Se presentan los conceptos básicos sobre la memoria principal y la jerarquía de memorias, las estrategias de asignación de memoria desde el heap (bloques de tamaño variable, bloques de tamaño fijo y *buddy system*), los mecanismos de protección por registros base/límite y segmentación, el código independiente de la posición, el esquema de paginado con sus tablas de páginas multinivel y, finalmente, los *Translation Lookaside Buffers* (TLB) como caché de traducciones. El objetivo es servir de material de estudio para preparar los prácticos asociados a la gestión de memoria y el segundo parcial.

Índice

1. Introducción	2
1.1. El ciclo de instrucción	2
2. Jerarquía de memorias	2
3. Asignación (<i>allocation</i>) de memoria	3
3.1. Heap con bloques de tamaño variable	3
3.2. Heap con bloques de tamaño fijo	4
3.3. El sistema compañero (<i>buddy system</i>)	5
4. Protección	5
4.1. Registros base y límite	5
4.2. Segmentación	6
5. Código independiente de la posición (PIC)	6
6. Paginado	7
6.1. Traducción de una dirección lógica	8
6.2. Espacios de direcciones por defecto	8
6.3. Páginas dispersas y heap allocator	8
6.4. Hardware vs. memoria direccionable	9
6.5. Páginas grandes y tablas multinivel	9
6.6. Caso de estudio: paginado en RISC-V RV-32	9
6.7. Formato de una <i>page table entry</i> en RV-32	10
7. Lookaside Translation Buffers (TLB)	10
7.1. La caché de traducciones	11
7.2. TLB con direct mapping	11
7.3. Otras configuraciones	12

1. Introducción

La **memoria principal** o RAM (*Random Access Memory*) es, desde el punto de vista del sistema, un arreglo de bytes en el que cada byte se identifica por su *dirección* o posición. Es una memoria **volátil**: el contenido se pierde cuando se corta la energía. La CPU interactúa directamente con la RAM en cada ciclo de ejecución de cada instrucción.

Idea intuitiva

En las arquitecturas *Von Neumann* las instrucciones del programa y los datos sobre los que opera *comparten* la misma memoria, a diferencia de la arquitectura Harvard que separa físicamente ambos espacios.

En un sistema multitarea la memoria física debe ser **particionada lógicamente** para separar las instrucciones y los datos de cada programa en ejecución. Esto le impone al SO dos responsabilidades inseparables:

- **Asignar** memoria al kernel y a cada uno de los procesos: decidir qué regiones de la RAM le tocan a quién, y cómo se liberan cuando ya no se usan.
- **Proteger** los espacios de memoria: garantizar que un proceso se ejecute *confinado*, es decir, que sólo pueda acceder a sus propias regiones y no pueda invadir las de otros procesos ni las del kernel.

1.1. El ciclo de instrucción

La CPU ejecuta repetidamente un ciclo con tres pasos:

1. **Fetch (lectura) de la instrucción.** Se carga en un registro interno la instrucción cuya dirección está en el *program counter* (pc).
2. **Decodificación.** Se identifica el tipo de instrucción y sus operandos.
3. **Ejecución** de la instrucción, que en general puede requerir *leer* o *escribir* en memoria.

Como se ve, todo paso del ciclo toca memoria; por eso la velocidad y la correcta gestión de la RAM determinan en gran medida el rendimiento del sistema.

2. Jerarquía de memorias

Las distintas memorias del sistema no son equivalentes: hay un *trade-off* clásico entre **velocidad** y **capacidad** (y costo). La CPU emplea *registros* internos para ejecutar las instrucciones; son las memorias más rápidas, pero hay muy pocos. Cada operación de lectura o escritura hacia/desde la RAM puede tomar varios ciclos de reloj, lo que obliga a la CPU a *frenarse* (*stall*) hasta que el dato esté disponible.

Para atenuar esta diferencia de velocidades se introducen las memorias **caché**: memorias de menor capacidad pero de mucha mayor velocidad de acceso que la RAM convencional, ubicadas entre los registros y la RAM.

Definición

Una memoria **caché** está diseñada para el acceso a los datos *por contenido* en lugar de por dirección. Cuando la CPU referencia una dirección de memoria:

- si el contenido de esa dirección ya está en la caché (*hit*), se recupera desde allí;
- si no se encuentra (*miss*), se accede a la RAM y se almacena en la caché para los accesos futuros.

Finalmente, para almacenar programas y datos en forma **permanente** se usan dispositivos secundarios como discos magnéticos, unidades de estado sólido u otros, que son los más lentos pero también los de mayor capacidad. La arquitectura define entonces una *jerarquía* de memorias ordenadas por velocidad de acceso (de mayor a menor):



Idea intuitiva

A medida que se asciende en la jerarquía (a la izquierda) la memoria es más rápida, más cara y de menor capacidad. La idea detrás de la caché es explotar la *localidad de referencia*: los programas tienden a acceder repetidamente a un conjunto pequeño de direcciones cercanas en el tiempo.

3. Asignación (*allocation*) de memoria

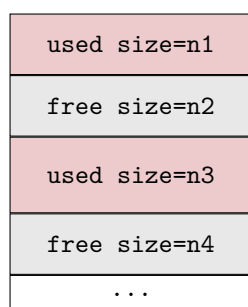
El kernel debe *asignar* y *liberar* bloques de memoria, tanto para los procesos de usuario como para sus propias estructuras de datos internas.

Una vez que el kernel se cargó en memoria durante el proceso de arranque (*boot*), reconoce el área de memoria libre disponible para asignar. Esta área se conoce como **heap**.

Sobre el heap se aplican dos grandes estrategias clásicas:

1. **Asignación de bloques de tamaño variable:** el heap se particiona lógicamente y se asignan bloques cuyo tamaño se ajusta al pedido.
2. **Asignación de bloques de tamaño fijo:** el heap se particiona en bloques todos del mismo tamaño y se asignan bloques completos (o varios contiguos) por cada pedido.

Variable size blocks



Fixed size blocks



3.1. Heap con bloques de tamaño variable

La primera estrategia se implementa habitualmente mediante una **lista de bloques libres** (*freelist*):

- Inicialmente existe un único bloque libre del tamaño total del heap.

- Ante una solicitud de un bloque de tamaño n , se busca en la lista un bloque libre b con $b.size \geq n$. Si $b.size > n$, se *particiona*: se asigna un bloque de tamaño n y queda un nuevo bloque libre b' con $b'.size = b.size - n$. Si no existe ningún bloque que satisfaga el pedido, se retorna error.
- Ante la liberación de un bloque, se inserta nuevamente en la lista de libres.

El algoritmo concreto que se use para buscar el bloque adecuado afecta directamente al rendimiento del sistema. Las tres estrategias clásicas son:

1. **First fit.** Se asigna el primer bloque encontrado con tamaño $\geq n$. Es la más rápida porque corta la búsqueda en cuanto encuentra un candidato.
2. **Best fit.** Se asigna el bloque que *mejor ajusta*, es decir, un bloque b con $b.size = m$, $m \geq n$ y para todo $b' \in freelist$ se cumple $b'.size \geq b.size$. Minimiza el desperdicio por pedido, pero suele dejar fragmentos pequeños y exige recorrer toda la lista.
3. **Worst fit.** Se asigna el bloque libre más grande. La idea es que el remanente sea lo suficientemente grande como para poder ser reutilizado.

Fragmentación externa. A medida que se hacen sucesivos pedidos y liberaciones, el heap puede ir **fragmentándose** en un gran número de bloques libres pequeños. Aunque la suma de esos bloques pueda ser grande, ningún bloque *individual* alcanza para satisfacer un pedido grande. Este problema se conoce como **fragmentación externa**.

Importante

La fragmentación externa es el costo principal del esquema de tamaño variable: el heap puede tener mucha memoria libre y aun así rechazar un pedido por no poder ofrecer un bloque contiguo suficientemente grande.

Semi-compactación. Una estrategia que alivia la fragmentación es: ante la liberación de un bloque, si quedan bloques libres *adyacentes*, fusionarlos o *compactarlos* en un único bloque libre de mayor tamaño. Esta técnica se conoce como **semi-compactación**.

3.2. Heap con bloques de tamaño fijo

En esta variante el heap se particiona en m bloques de un tamaño fijo s .

Ante un requerimiento de un bloque de tamaño n :

- Si $n \leq s$, se asigna cualquier bloque libre.
- Si $n > s$, se debe asignar el mínimo número k de bloques *contiguos* tales que $k \times s \geq n$.

Una ventaja muy importante en términos de uso de memoria es que las listas (o conjuntos) de bloques libres y usados pueden representarse como un **bit vector**: un bit por bloque, encendido si está asignado y apagado si está libre.

La elección del tamaño s . El tamaño s es *crítico*:

- Si s es grande, se agrava la **fragmentación interna**: cada pedido recibe un bloque completo, pero el remanente $s - n$ no se aprovecha.
- Si s es pequeño, una asignación grande requiere encontrar varios bloques libres *contiguos*; si esa contigüidad no existe, el pedido no se puede satisfacer aunque haya muchos bloques libres dispersos.

Idea intuitiva

La fragmentación interna es “desperdicio dentro del bloque asignado” (memoria que el proceso recibió pero no usa). La fragmentación externa es “desperdicio entre bloques” (memoria libre que no se puede asignar porque no es contigua). Ambos esquemas tienen sus contraindicaciones.

3.3. El sistema compañero (*buddy system*)

Para combinar las ventajas de ambas estrategias se diseñan soluciones *híbridas*. El **buddy system** es el ejemplo clásico.

La idea es particionar el heap en bloques cuyos tamaños sean *potencias de 2*, desde un mínimo 2^l hasta un máximo 2^h . Por ejemplo, con $l = 16$ y $h = 20$ los bloques posibles son de 64 KB, 128 KB, 256 KB, 512 KB y 1024 KB.

Algoritmo de asignación.

1. Inicialmente, el heap se particiona en bloques del tamaño máximo 2^h (1024 KB en el ejemplo).
2. Ante un requerimiento de tamaño n :
 1. Seleccionar un bloque libre b con mejor ajuste.
 2. Si $b.size \geq 2n$, dividir b en dos bloques b_0 y b_1 de tamaño $b.size/2$. Tomar $b = b_0$ y repetir este paso mientras $b.size \geq 2n$ (y siempre que $b.size > 2^l$).
3. Asignar el bloque b .

Liberación. Cuando se libera un bloque se aplica el algoritmo a la inversa: si el bloque *compañero* (*buddy*) contiguo es del mismo tamaño y también está libre, ambos se unen en un único bloque del doble de tamaño, mientras esto se pueda repetir y sin pasar de 2^h .

Importante

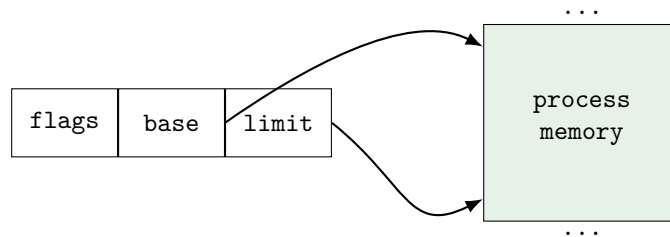
El buddy system es un compromiso entre fragmentación interna y externa: mantiene bloques de tamaño potencia de 2 (lo que simplifica la contabilidad y la fusión), pero al mismo tiempo permite que el heap se particione dinámicamente según los pedidos. Es la base de muchos *allocators* de kernel reales (por ejemplo el de Linux).

4. Protección

Asignar memoria es sólo la mitad del problema: el SO también debe garantizar que cada proceso **sólo pueda acceder a su área de memoria**. Supongamos que un proceso recibe su memoria de forma *contigua*; entonces basta con que el hardware pueda *detectar* todo acceso fuera de rango.

4.1. Registros base y límite

El mecanismo más simple consiste en un registro de control que contiene la **dirección base** y la **dirección límite** del área de memoria asignada al proceso. Además puede incluir **flags** de *permisos de acceso*: lectura, escritura, ejecución, etc.



Cuando el SO realiza un *context switch* a un proceso, carga las direcciones correspondientes en estos registros. La CPU, en cada acceso a memoria, verifica que la dirección referenciada caiga en el rango permitido:

$$base \leq address \leq limit.$$

Memory fault exception. Si una instrucción referencia una dirección fuera de los límites, o si la operación de lectura/escritura no respeta los permisos definidos en los flags, la CPU genera una **memory fault exception**. El SO atraparé la excepción con algún *exception handler* y, si no es recuperable, eliminará al proceso. Este error es exactamente el conocido **segmentation fault** de los sistemas tipo UNIX.

4.2. Segmentación

El esquema de un único par base/límite es muy rígido: obliga a que toda la memoria del proceso esté en un bloque contiguo, lo que no siempre se puede lograr en un sistema fragmentado.

La **segmentación** extiende la idea: se utilizan *varios* pares base/límite, uno por cada *segmento* lógico del proceso (*texto*, *datos*, *stack*, *heap*, etc.). Así cada segmento puede vivir en una región de memoria distinta y los segmentos no necesitan ser físicamente contiguos entre sí.

Idea intuitiva

A modo de ejemplo, la arquitectura Intel IA-32 (x86) soporta varios registros de segmento (base + límite): **CS** (*code segment*), **DS** (*data segment*), **SS** (*stack segment*) y otros como **ES**, **GS**, **FS**.

5. Código independiente de la posición (PIC)

Con la maquinaria de protección por segmentos aparece otro beneficio: el código de un programa puede ejecutarse *independientemente* de la dirección base donde se cargó. Esto se logra con **código independiente de la posición** (*position-independent code*, **PIC**).

Definición

En el código PIC, las direcciones de memoria que aparecen como operandos de las instrucciones no son *absolutas* sino **relativas** a una dirección base contenida en un registro de segmento.

Los registros de segmento contienen las direcciones base; la CPU *transforma* cada dirección relativa en una **dirección física** sumándole la dirección base correspondiente al segmento.

Notación en x86. En x86, una dirección de memoria en una instrucción se denota como **segment:offset**. Algunos casos:

- Un salto `jmp address` es equivalente a `jmp cs:address`.
- Una instrucción de lectura/escritura desde/hacia memoria como `mov %eax, address` es equivalente a `mov %eax, ds:address`.

- Las instrucciones sobre la pila (**push** / **pop**) refieren por omisión al segmento **ss**.

Idea intuitiva

PIC es lo que permite, por ejemplo, que el mismo binario de una biblioteca compartida (*shared library*) se pueda cargar en una dirección distinta en cada proceso sin tener que reescribir su código. También es la base del esquema ASLR moderno (*Address Space Layout Randomization*).

6. Paginado

La segmentación resuelve el problema de no necesitar bloques contiguos grandes para todo el proceso, pero todavía sufre de fragmentación externa al manejar segmentos enteros. La solución estándar de los SO modernos es el **paginado**.

Definición

El **paginado** consiste en particionar la memoria física en bloques pequeños del *mismo tamaño*, llamados **páginas** o *frames*, típicamente de 4 KB. Cada espacio de direcciones lógicas de un proceso (y del kernel) se representa como un conjunto de páginas que pueden vivir en cualquier ubicación física.

page 0	0
page 1	4096
...	8192
page n-1	max address

Con paginado es posible definir un **espacio de memoria** para cada proceso (y el kernel) como el conjunto de páginas a las que puede acceder. Estos conjuntos se representan en memoria como arreglos y se conocen como **tablas de páginas** (*page tables*).

Direcciones lógicas vs físicas. Las páginas asignadas a un proceso no necesitan estar *físicamente contiguas*, aunque para el proceso aparecen como si lo estuvieran. Esto requiere de un mecanismo de *traducción* de direcciones **lógicas o virtuales** a direcciones **físicas o reales**. Formalmente, se necesita una función

$$map : va \rightarrow pa,$$

donde *va* es una dirección lógica (*virtual address*) y *pa* es la dirección física correspondiente.

El hardware con soporte de paginado implementa esta función mediante una *tabla de páginas* que representa la traducción *paging* : $va \rightarrow pa$. Cada entrada de la tabla se llama **page table entry** (**pte**) y básicamente es un par (*ppn*, *flags*), donde:

- **ppn** (*physical page number*) es el número de página física correspondiente (las páginas se numeran desde 0);
- **flags** son permisos de acceso y bits de control.

Por cada proceso y para el kernel, el SO construye su propio espacio de memoria asociando una tabla de páginas distinta.

6.1. Traducción de una dirección lógica

Importante

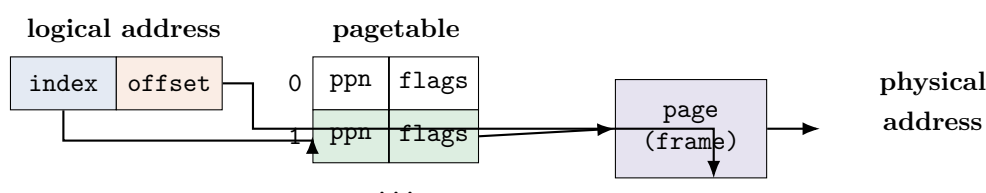
En paginado, una dirección que aparece en una instrucción del programa es, en realidad, una *dirección lógica* y generalmente se interpreta como un par (*index*, *offset*).

- El **index** (*virtual page number*, *vpn*) identifica la entrada de la tabla de páginas asociada al proceso.
- El **offset** es el desplazamiento *dentro* de la página.

La dirección física se obtiene como

$$pa = pagetable[index].ppn \times PS + offset,$$

donde *PS* es el tamaño de página (*page size*).



Tamaño del offset. El offset contiene tantos bits como sea necesario según el tamaño de página. Por ejemplo, si el tamaño de página es 4096 bytes (2^{12}), el offset necesita 12 bits.

Page fault. La CPU genera una excepción de tipo **page fault** si una dirección lógica indexa en una *pte* sin permisos de acceso o inválida (por ejemplo, una entrada cuyo bit *V* está apagado).

6.2. Espacios de direcciones por defecto

Con paginado es posible que cada proceso tenga, por ejemplo, un espacio de direcciones de la forma $[0, size]$ donde *size* es la memoria requerida por el proceso. Así todos los programas se pueden compilar con opciones de memoria *por default* y pueden coexistir en memoria *referenciando las mismas direcciones lógicas*: en tiempo de ejecución cada una se traducirá a direcciones físicas distintas.

Idea intuitiva

Dos procesos pueden leer la dirección lógica 0x00001000 sin pisarse: la tabla de páginas de cada uno mapea ese índice a frames físicos distintos. El hardware se encarga, transparentemente, de que cada proceso “vea” su propio espacio de memoria.

6.3. Páginas dispersas y heap allocator

Cuando un proceso usa paginado, un bloque de memoria con páginas *lógicamente contiguas* no necesariamente se corresponde con páginas *físicamente contiguas*. Esto permite que un **heap allocator** le asigne páginas al proceso con cualquier dirección física, mientras quede mapeada a una dirección lógica del proceso.

A modo de ejemplo, un proceso podría tener:

- 2 páginas (8 KB) para el área de *código* (texto);
- 1 página para *datos*;
- 1 página para el *stack*;
- un *hueco* de páginas inválidas en el medio para detectar **stack overflow**.

Las páginas de código y datos resultan lógicamente contiguas, pero físicamente pueden estar dispersas por toda la memoria.

Ejemplo numérico. Asumiendo direcciones de 32 bits con los 20 bits más significativos como *index* y los 12 menos significativos como *offset*, en páginas de 4 KB:

Si el espacio de direcciones lógicas es $[0, sz = 0x5000]$ (20 KB), una instrucción que referencie la dirección lógica `0x00002004` (área de datos) se interpreta con `index = 2` y `offset = 0x004`. Si la *pte* de la entrada 2 contiene `ppn = 11`, la dirección física final es:

$$11 \times 4096 + 4 = 0xb004.$$

6.4. Hardware vs. memoria direccionable

Las tablas de páginas deberían tener suficientes entradas como para *barrer* toda la memoria direccionable por el sistema (esto es crítico en el kernel, que debe poder acceder a toda la memoria).

Importante

Es común que el hardware tenga *menos RAM física* que el espacio de direcciones lógicas posibles. Por ejemplo, en el modelo **RISC-V RV39** la CPU es de 64 bits (espacio de direcciones $[0, 2^{64}]$), pero el bus físico de direcciones es de sólo 39 bits (500 GB).

6.5. Páginas grandes y tablas multinivel

Con espacios de direcciones grandes (por ejemplo 4 GB en una arquitectura de 32 bits) el número de entradas en una tabla de páginas puede ser enorme. Representar una tabla plana con tantas entradas requeriría asignar un bloque de memoria física contigua de gran tamaño, algo costoso o directamente inviable.

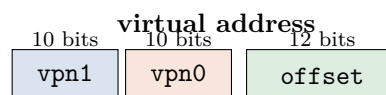
Para mitigarlo existen dos herramientas combinables:

1. **Páginas más grandes.** El hardware moderno permite definir tablas que refieran a páginas de mayor tamaño (4 MB o más). Esto reduce la cantidad total de entradas pero a costa de aumentar la *fragmentación interna*: cada página desperdicia más memoria si no se llena por completo.
2. **Tablas multinivel.** Aún con páginas pequeñas, los procesos suelen dejar muchos *huecos* en su espacio de direcciones lógicas, lo que en una tabla plana sería desperdicio. Por eso las tablas de páginas se representan como **árboles de n niveles**.

Cada nodo del árbol tiene el tamaño de una página. Sólo se reservan los nodos correspondientes a regiones efectivamente usadas del espacio de direcciones; las regiones vacías no consumen memoria.

6.6. Caso de estudio: paginado en RISC-V RV-32

En la arquitectura **RISC-V RV-32** el espacio de direcciones es de 32 bits (4 GB) y la tabla de páginas tiene la forma de un **árbol de dos niveles**. El registro `satp` (*Supervisor Address Translation and Protection*), de acceso privilegiado, contiene el número de página física de la raíz del árbol.



Una dirección lógica se divide en tres partes:

- **vpn1** (10 bits más significativos): índice o *virtual page number* al *nodo raíz* del árbol. Su **pte** contiene el **ppn** del nodo hijo.
- **vpn0** (10 bits siguientes): índice en el nodo hoja. Su **pte** contiene el **ppn** de la página o *frame* destino.
- **offset** (12 bits menos significativos): desplazamiento dentro de la página destino.

Ejemplo. La dirección lógica 0x0000100f se interpreta como:

vpn1	vpn0	offset
0000000011	0000000001	000000001111

La traducción procede así:

1. Comienza en el nodo raíz en la entrada **vpn1** = 3 (tercera entrada).
2. Suponiendo que esa **pte** contiene un **ppn** que lleva al nodo hijo (digamos *y*), se indexa en el nodo hoja *y* en la entrada **vpn0** = 1.
3. En esa entrada el **ppn** = *k* lleva a la dirección base física del frame destino.
4. La dirección física final es la dirección base del frame sumada al offset (0x00f).

Formalmente, la función de traducción es

$$pa = (satp.ppn[va.vpn1].ppn \times PS)[va.vpn0].ppn \times PS + va.offset.$$

6.7. Formato de una *page table entry* en RV-32

31	20	19	10	9	flags	0
ppn					DAGUXWRV	

Los *flags* en una **pte** son los siguientes bits:

- **V** (*Valid*): indica si la entrada es válida.
- **D** (*Dirty*): encendido por el hardware cuando la página correspondiente fue *modificada* (escrita) por una instrucción.
- **A** (*Accessed*): encendido por el hardware cuando la página fue accedida para lectura o escritura.
- **U** (*User mode*): permiso de acceso en modo usuario.
- **R, W, X**: permisos de *Read*, *Write* y *eXecute* sobre la página.
- **G** (*Global mapping*): la entrada existe en todas las tablas (mapping global).

En el proceso de traducción de una dirección lógica a una física, la CPU verifica los flags correspondientes. Si hay una violación (por ejemplo, $V = 0$, o el acceso no respeta los permisos *R/W/X/U*), se genera un *page fault*. El kernel atrapa estas excepciones y actúa en consecuencia.

Context switch. El kernel debe cambiar el registro **satp** en cada context switch para que apunte a la tabla de páginas de la nueva tarea seleccionada. Los bits *D* y *A* son la base de muchas políticas de *memoria virtual* que se ven en el capítulo siguiente.

7. Lookaside Translation Buffers (TLB)

Un sistema con paginado traduce *constantemente* direcciones lógicas a físicas usando las tablas de páginas. Estas tablas están almacenadas en RAM y son configuradas por el SO.

Importante

Sin ninguna optimización, en un esquema multinivel *cada acceso a memoria del programa requiere varios accesos extra a RAM*: uno por cada nivel del árbol de tablas (por ejemplo, *pte* raíz, *pte* de la hoja) y finalmente el acceso al dato. En RV-32 esto multiplicaría por 3 cada referencia.

7.1. La caché de traducciones

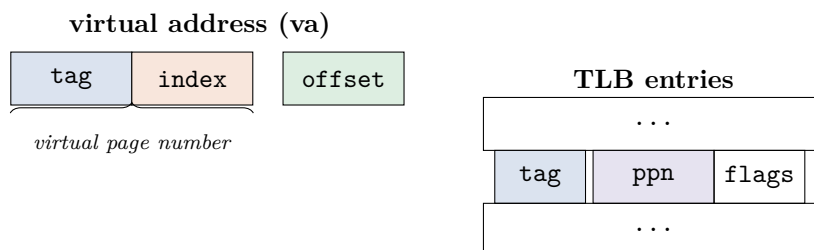
Para evitar esta sobrecarga las CPUs modernas usan una memoria caché especial dedicada a almacenar las *ptes* más frecuentemente referenciadas. Esta caché se conoce como **Translation Lookaside Buffer (TLB)**. La TLB reside físicamente entre la CPU y la caché de nivel 1 (L1).

Definición

La TLB es una memoria **direccionable por contenido**: se busca por dirección lógica y, si está presente (*hit*), retorna la dirección física correspondiente; en caso contrario ocurre un *miss* y la CPU debe consultar las tablas de páginas en RAM.

7.2. TLB con direct mapping

Generalmente, una TLB implementa una **función hash** donde la clave de búsqueda son una parte de los bits *menos significativos* del *virtual page number* (vpn), asociados a un **tag** que es el conjunto de bits *más significativos* del vpn. El par (**tag**, **index**) identifica unívocamente a un vpn.



Cada entrada de la TLB contiene el **tag**, el **ppn** (*physical page number*) y los **flags**, entre los cuales está el bit **valid (V)** que indica si la entrada es válida.

Condición de *hit*. La caché así descrita es **1-way associative cache** (direct mapping): hay una sola **ppn** posible para cada **index**. Sea **va.tag** y **va.index** las partes de la dirección virtual recibida; ocurre un *hit* cuando:

$$TLB[index].tag == va.tag \quad \wedge \quad TLB[index].flags \& BIT_V \neq 0,$$

y **TLB[index].ppn** contiene la **physical page number** buscada.

Manejo de un *miss*. Si ocurre un *miss*, la CPU accede a la tabla de páginas en RAM (o a una caché de nivel inferior) y obtiene la **ppn** correspondiente. Luego se setea **TLB[index]** apropiadamente con los datos de la nueva traducción para acelerar accesos futuros.

Colisiones. Inevitablemente, varias direcciones virtuales distintas comparten el mismo **index** pero con **tag** diferente. Eso produce una **colisión**: la entrada anterior es reemplazada con el nuevo trío **<tag, ppn, flags>**.

7.3. Otras configuraciones

Existen organizaciones más sofisticadas que permiten almacenar *más de una entrada* (un *set*) en el mismo *index*; son las *set-associative caches* (o *associative caches*). La configuración *1-way* descrita arriba es el caso más simple. A mayor asociatividad menor tasa de colisiones, pero mayor complejidad y costo del hardware de búsqueda.

Algunas arquitecturas, como **MIPS**, permiten que sea el *kernel* el que decida (por software) qué hacer ante un *miss*: el hardware genera una excepción (trap) y el kernel implementa la política de reemplazo y la carga de la nueva entrada.

8. Cierre y conexión con memoria virtual

A lo largo del capítulo se construye, paso a paso, la maquinaria que hace posible la gestión moderna de memoria:

- La **jerarquía de memorias** (registros, cachés, RAM, disco) explica por qué la velocidad de acceso a la memoria es un factor crítico y motiva la existencia de toda la maquinaria de caching y traducción.
- Las estrategias clásicas de **asignación** (tamaño variable con *first/best/worst fit*, tamaño fijo y *buddy system*) muestran el compromiso permanente entre *fragmentación interna* y *externa*.
- Los mecanismos de **protección** (registros base/límite, segmentación) garantizan que cada proceso quede *confinado* a su espacio de memoria y son el origen del clásico *segmentation fault*.
- El **código independiente de la posición** (PIC) hace que un mismo binario pueda ejecutarse en cualquier dirección base, lo que habilita bibliotecas compartidas y técnicas modernas como ASLR.
- El **paginado** unifica todo lo anterior: divide la memoria física en frames del mismo tamaño y le da a cada proceso (y al kernel) un espacio de direcciones lógicas independiente. Las tablas de páginas multinivel evitan el desperdicio de las tablas planas en espacios de direcciones grandes.
- Las **TLBs** aceleran la traducción dirección lógica → dirección física, evitando que cada acceso a memoria pague el costo de recorrer varios niveles de tablas en RAM.

Idea intuitiva

Esta maquinaria es exactamente la base sobre la que se construyen las políticas de **memoria virtual** (swapping, demand paging, copy-on-write, mapeo de archivos, ...) que se estudian en el capítulo siguiente. Los bits V, D y A de la *pte*, y la posibilidad de generar *page faults*, son las herramientas que el SO necesita para implementar esas políticas.